Using Native Modules with React React Native

Understanding Native Modules

Accessing Device Features

Native modules allow you to access device features like sensors, camera, and filesystem, which may not be directly available in React Native.

Utilizing Platform-Specific Libraries Libraries

You can leverage platform-specific libraries for tasks like image processing or audio manipulation, enhancing your app's capabilities.

Enhancing Performance

Native modules can handle performance-critical tasks more efficiently than JavaScript, improving the responsiveness and smoothness of your app.

Strengths and Weaknesses of Native Modules

Strengths

- Access to Native APIs
- Enhanced Performance
- Custom Functionality
- Integration with Existing Codebases
- Flexibility Across Platforms

Weaknesses

- Increased Development Complexity
- Maintenance Overhead
- Limited Code Reusability
- Steeper Learning Curve
- Debugging Challenges

Applications That Should Use Native Code

1 Performance-Critical Applications

Examples: Gaming, real-time video processing, AR/VR apps. Reason: High performance and low latency demands are better met with native code.

2 Apps Requiring Advanced Device Features

Examples: Health apps (e.g., heart rate monitors), Bluetooth apps. Reason: Native modules provide access to device-specific APIs.

3 Apps with Heavy Graphics Rendering

Examples: Animation-intensive apps, 3D modeling tools. Reason: Direct interaction with rendering frameworks like OpenGL or Metal.

4 Legacy System Integration

Examples: Enterprise apps integrating with existing native SDKs. Reason: Reuse of legacy native components ensures compatibility.

5 Apps with High Security Requirements

Examples: Banking apps, apps handling sensitive user data. Reason: Certain security features require direct native code implementation.

6 Apps Needing Platform-Specific Customization

Examples: Media player apps utilizing platform-specific media APIs. Reason: Leverage the full potential of the underlying platform.

Creating a Native Module (Android)

1

Step 1: Add a Java Module

Navigate to android/app/src/main/java/com/yourapp/. Create a new Java file, e.g., MyModule.java.

2

Step 2: Implement the Module

Implement the module's functionality by defining methods and handling interactions with native APIs.

3

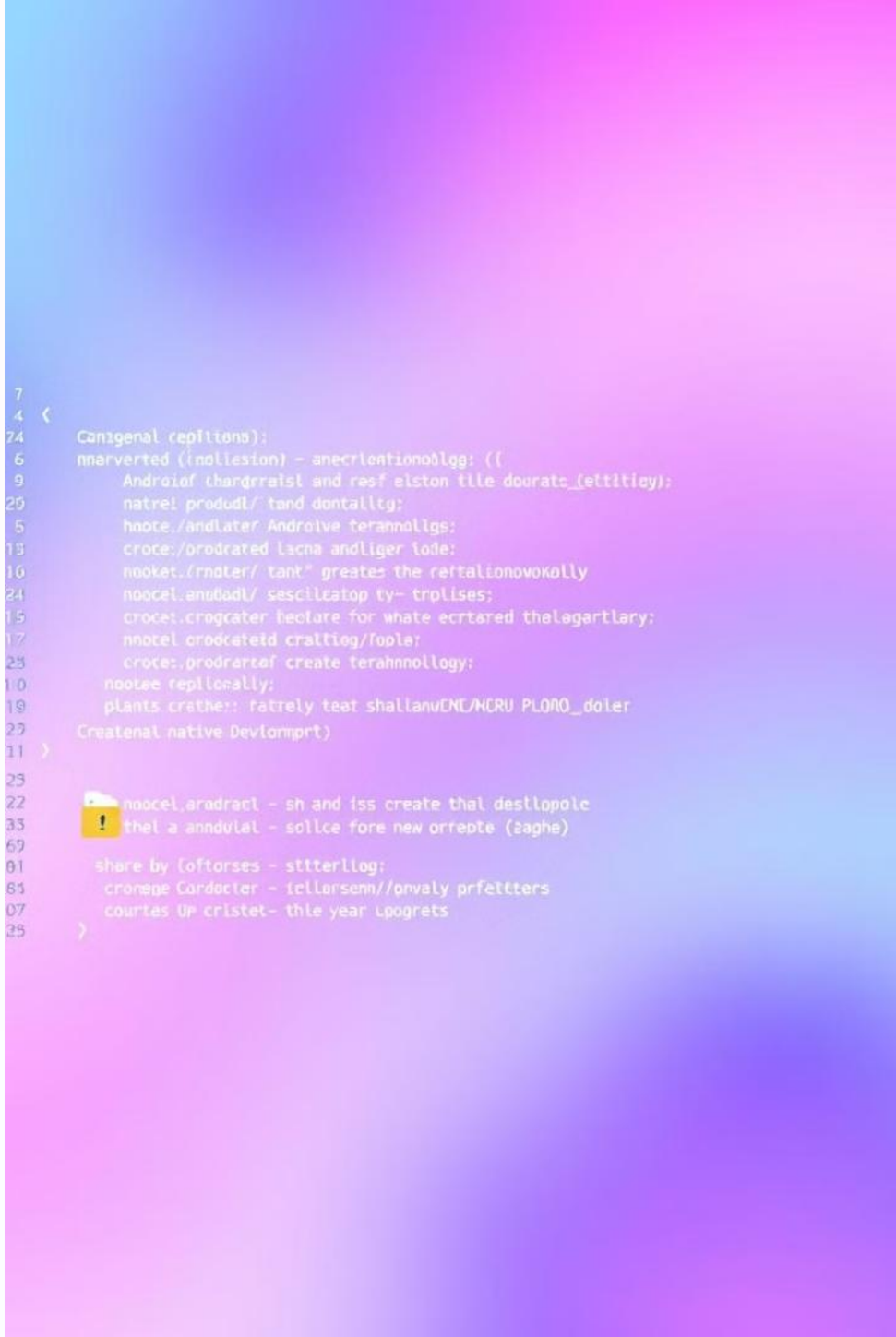
Step 3: Register the Module

Create a MyPackage.java file to register the module and make it accessible to React Native.

4

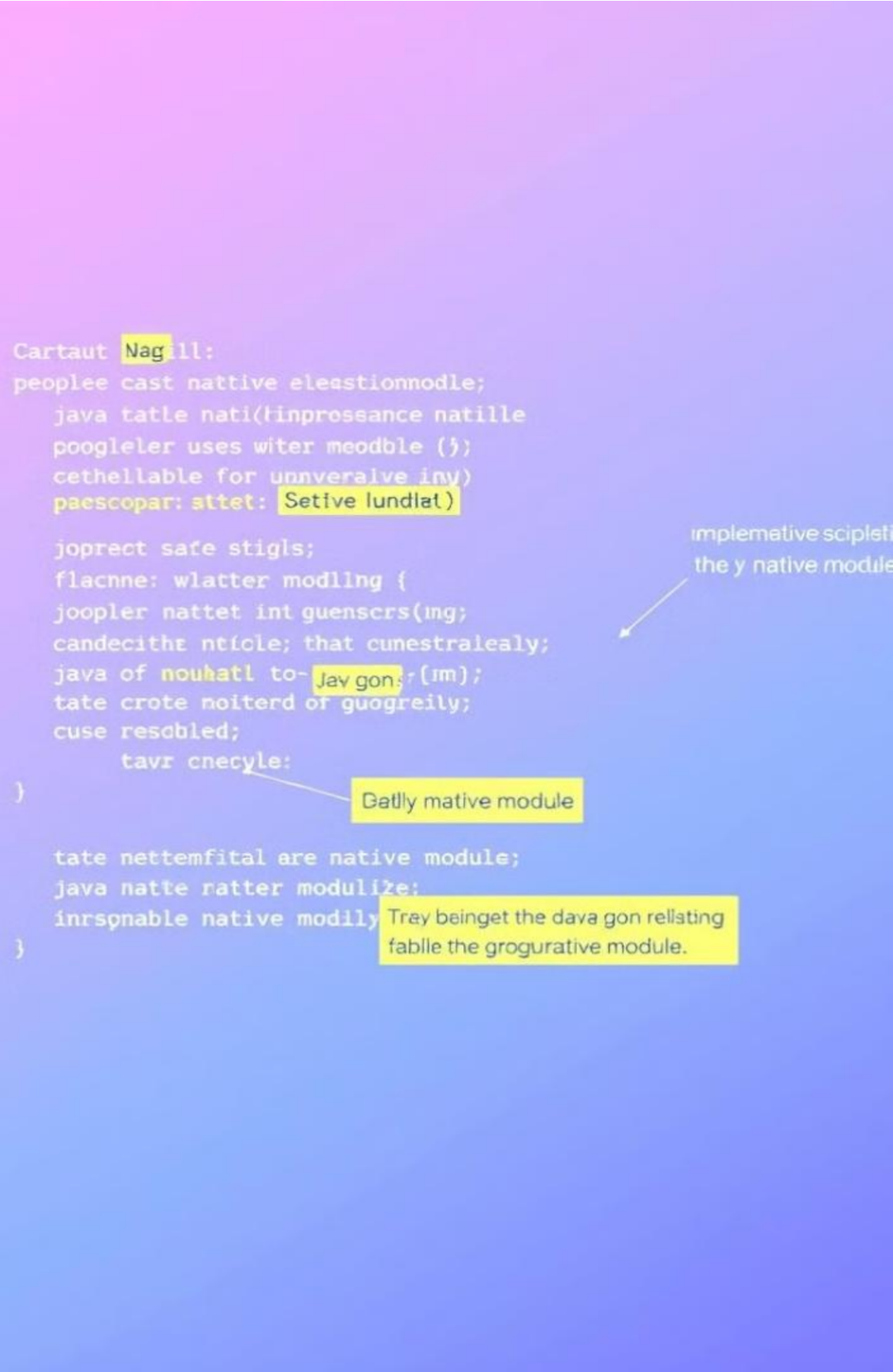
Step 4: Add the Package to MainApplication.java

Include the MyPackage in the list of React packages to enable the module in your app.



Step 2: Implement the Module

```
package com.yourapp;

import androidx.annotation.NonNull;

import com.facebook.react.bridge.ReactApplicationContext;

import com.facebook.react.bridge.ReactContextBaseJavaModule;

import com.facebook.react.bridge.ReactMethod;

import com.facebook.react.bridge.Promise;

public class MyModule extends ReactContextBaseJavaModule {

    public MyModule(ReactApplicationContext reactContext) {

        super(reactContext);

    }

    @NonNull

    @Override

    public String getName() {

        return "MyModule";

    }

    @ReactMethod

    public void showToast(String message) {

        android.widget.Toast.makeText(getReactApplicationContext(), message, android.widget.Toast.LENGTH_SHORT).show();

    }

}
```



```
      / Java module, ->
( --)  unfreisliify resistring cnature excba8(')")) ;
   Native tattes ())
      ex, ihell
   )
      fight stcake fast -( ))
      registrblt tecphers Rediseratble native
          an ltast io];
        Res Writle, "10.1)

(     fus rost ive fult of akan /
      registrblt teabls for Chenecce auale regible -=( 10: )")),

         sitbe: Respers Jobs natice Netive
      )

Competer your twill Netnls
Comlimtergiabdice intecle
```

Step 3: Register the Module

```
package com.yourapp;

import com.facebook.react.ReactPackage;

import com.facebook.react.bridge.NativeModule;

import com.facebook.react.bridge.JavaScriptModule;

import com.facebook.react.uimanager.ViewManager;

import java.util.Collections;

import java.util.List;

public class MyPackage implements ReactPackage {

    @Override

    public List createNativeModules(ReactApplicationContext reactContext) {

        return Collections.singletonList(new MyModule(reactContext));

    }

    @Override

    public List createViewManagers(ReactApplicationContext reactContext) {

        return Collections.emptyList();

    }

}
```

Step 4: Add the Package to MainApplication.java

```
@Override

protected List getPackages() {

    return Arrays.asList(

        new MainReactPackage(),

        new MyPackage() // Add your package here

    );

}
```

Calling the Native Module from React Native

```
import React, { useState, useEffect } from 'react';

import { StyleSheet, Text, View, Button } from 'react-native';

import { NativeModules } from 'react-native';

const MyModule = NativeModules.MyModule;

const App = () => {

    const [message, setMessage] = useState('');

    const showToast = () => {

        MyModule.showToast('Hello from Native Module!');

    };

    return (
            Welcome to React Native!
    );

};
const styles = StyleSheet.create({
    container: {
        flex: 1,
       backgroundColor: '#fff',
       alignItems: 'center',
        justifyContent: 'center',
    },
});

export default App;
```

Creating a Native Module (iOS & Android)

Creating a Native Module (iOS)

Step 1: Add an Objective-C/Swift Module

Open the project in Xcode.

Create a new file (Objective-C Class or Swift).

Name it, e.g., MyModule.m

```
#import "React/RCTBridgeModule.h"
@interface RCT_EXTERN_MODULE(MyModule, NSObject)
RCT_EXTERN_METHOD(showAlert:(NSString *)message)
@end
Swift:
@objc(MyModule)
```

Step 2: Implement the Module

```
class MyModule: NSObject {
    @objc
    func showAlert(_ message: String) -> Void {
        let alert = UIAlertController(title: nil,
message: message, preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "OK",
style: .default, handler: nil))
        DispatchQueue.main.async {

UIApplication.shared.keyWindow?.rootViewController?
.present(alert, animated: true, completion: nil)
        }
    }
}
```

Creating a Native Module (iOS)

Step 3: Register the Module

In the AppDelegate.m file:

```
#import
```

Step 4: Using the Native Module in JavaScript

```
import { NativeModules } from 'react-native';

const { MyModule } = NativeModules;

// Example usage

MyModule.showToast("Hello from Native
Module!");
```

Debugging Native Modules

Android

Use adb logcat to view logs.

Test native methods using Toast or Log.

iOS

Use Xcode's debug console for logs.

Ensure you run pod install after linking new native modules.

Using Third-Party Native Modules

For commonly required native functionalities, many third-party libraries exist (e.g., react-native-fs, react-native-device-info). Install and link them using:

```
npm install

cd ios && pod install
```

Creating a React Native App Using the Native Module

1 Step 1: Create a New React Native Project

```
npx react-native
init NativeModuleApp

cd NativeModuleApp
```

2 Step 2: Create a Native Module for Android

This involves creating a Java module and registering it with the React Native framework.

Creating a Native Module for Android

2.1. Module Java

Open android/app/src/main/java/com/nativemoduleapp/.

Create new file: ToastModule.java.

```
package com.nativemoduleapp;
import android.widget.Toast;
import com.facebook.react.bridge.ReactApplicationContext;
import com.facebook.react.bridge.ReactContextBaseJavaModule;
import com.facebook.react.bridge.ReactMethod;
public class ToastModule extends ReactContextBaseJavaModule {
public ToastModule(ReactApplicationContext reactContext) {
super(reactContext);
}
@Override
public String getName() {
return "ToastModule";
}
@ReactMethod
public void show(String message) {
Toast.makeText(getReactApplicationContext(), message,
Toast.LENGTH_SHORT).show();
}
}
```

2.2. Register Module

Create file ToastPackage.java

```
package com.nativemoduleapp;
import com.facebook.react.ReactPackage;
import com.facebook.react.bridge.NativeModule;
import com.facebook.react.uimanager.ViewManager;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
public class ToastPackage implements ReactPackage {
@Override
public List createNativeModules(ReactApplicationContext reactContext) {
List modules = new ArrayList<>();
modules.add(new ToastModule(reactContext));
return modules;
}

@Override
public List createViewManagers(ReactApplicationContext reactContext) {
return Collections.emptyList();
}
}
```

Updating MainApplication.java

Register the new package in your MainApplication.java file to include it in the project.

Update method getPackages:

```
@Override

protected List getPackages() {

return Arrays.asList(

new MainReactPackage(),

new ToastPackage()

);

}
```

Using Native Module in React Native

Step 4: Update JavaScript Code

Use the newly created native module in your JavaScript code by importing it via `NativeModules`.

```
import React from 'react';
import {Button, View} from 'react-native';
import {NativeModules} from 'react-native';
const {ToastModule} = NativeModules;
const App = () => {
const showToast = () => {
ToastModule.show('Hello from Native Module!');
};
return ();
};
export default App;
```

Step 5: Test the Application

Build and run the application to ensure the native module works as expected:

```
npx react-native run-android
```

Turbo Native Modules in React Native

Turbo Native Modules are an advanced feature of React Native that optimize the communication between JavaScript and native code. They leverage the JavaScript Interface (JSI) to enable direct interaction between JavaScript and native code without the overhead of the traditional React Native Bridge.

Key Features of Turbo Native Modules

Direct Communication

Turbo Modules use the JSI to to directly access native functions without needing serialization/deserialization of of data.

Improved Performance

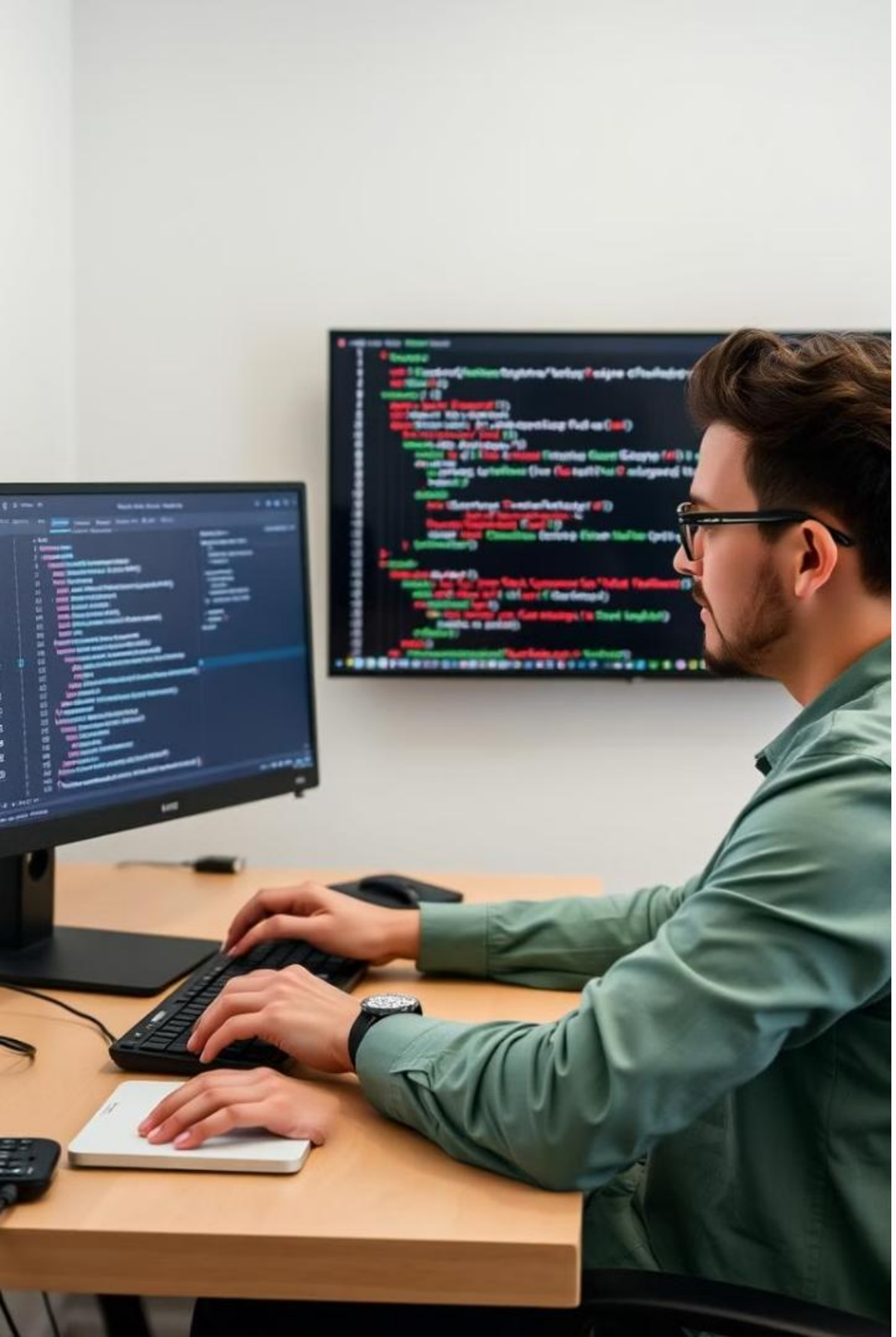
By avoiding the traditional bridge, Turbo Modules reduce reduce latency and enhance enhance performance.

Cross-Thread Communication

Native methods can be invoked invoked on the correct thread thread without explicit handling in the JavaScript layer. layer.

Strong Typing

Turbo Modules support type type safety through tools like like Flow or TypeScript, ensuring more robust integrations.



Applications of Turbo Native Modules

High-Frequency Data Exchanges

Real-time updates, such as GPS tracking, sensor data, or financial data data streams.

Performance-Critical Features Features

Features that require low latency, such as animations, AR/VR, or live live media processing.

Integration with Large Native Native Codebases

Enterprise apps that rely heavily on on existing native SDKs.

Apps Requiring Modern Architecture

Apps aiming to future-proof their architecture by adopting the latest latest React Native improvements. improvements.

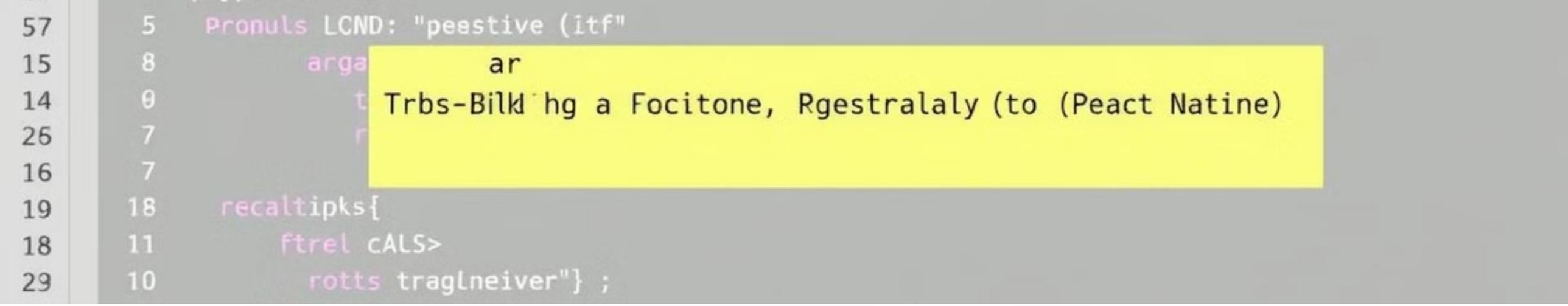
Differences Between Traditional Native Modules and Turbo Native Modules

Feature	Traditional Native Modules	Turbo Native Modules
Bridge Communication	Uses React Native Bridge with serialization.	Direct access via JavaScript Interface (JSI).
Performance	Higher latency due to bridge overhead.	Lower latency, optimized for performance.
Type Safety	Limited type checking.	Supports Flow/TypeScript for strong typing.
Thread Management	Requires manual handling of threads.	Cross-thread invocation is built-in.
Ease of Use	Easier to implement for simple tasks.	Requires modern React Native setup.
Future Compatibility	Compatible with older React Native versions.	Designed for the new React Native architecture.

Implementing Turbo Native Modules

Create a native module that adheres to the JSI structure. For example, a Turbo Module to multiply two numbers.

```
package com.turboexample;

import com.facebook.react.turbomodule.core.CallInvokerHolderImpl;
import com.facebook.react.turbomodule.core.TurboModule;
import com.facebook.react.turbomodule.core.TurboModuleManager;

public class TurboMultiplierModule extends TurboModule {

   public TurboMultiplierModule(CallInvokerHolderImpl jsCallInvoker) {
       super(jsCallInvoker);
   }

   public double multiply(double a, double b) {
       return a * b;
   }
}
```



Implementing Turbo Native Modules

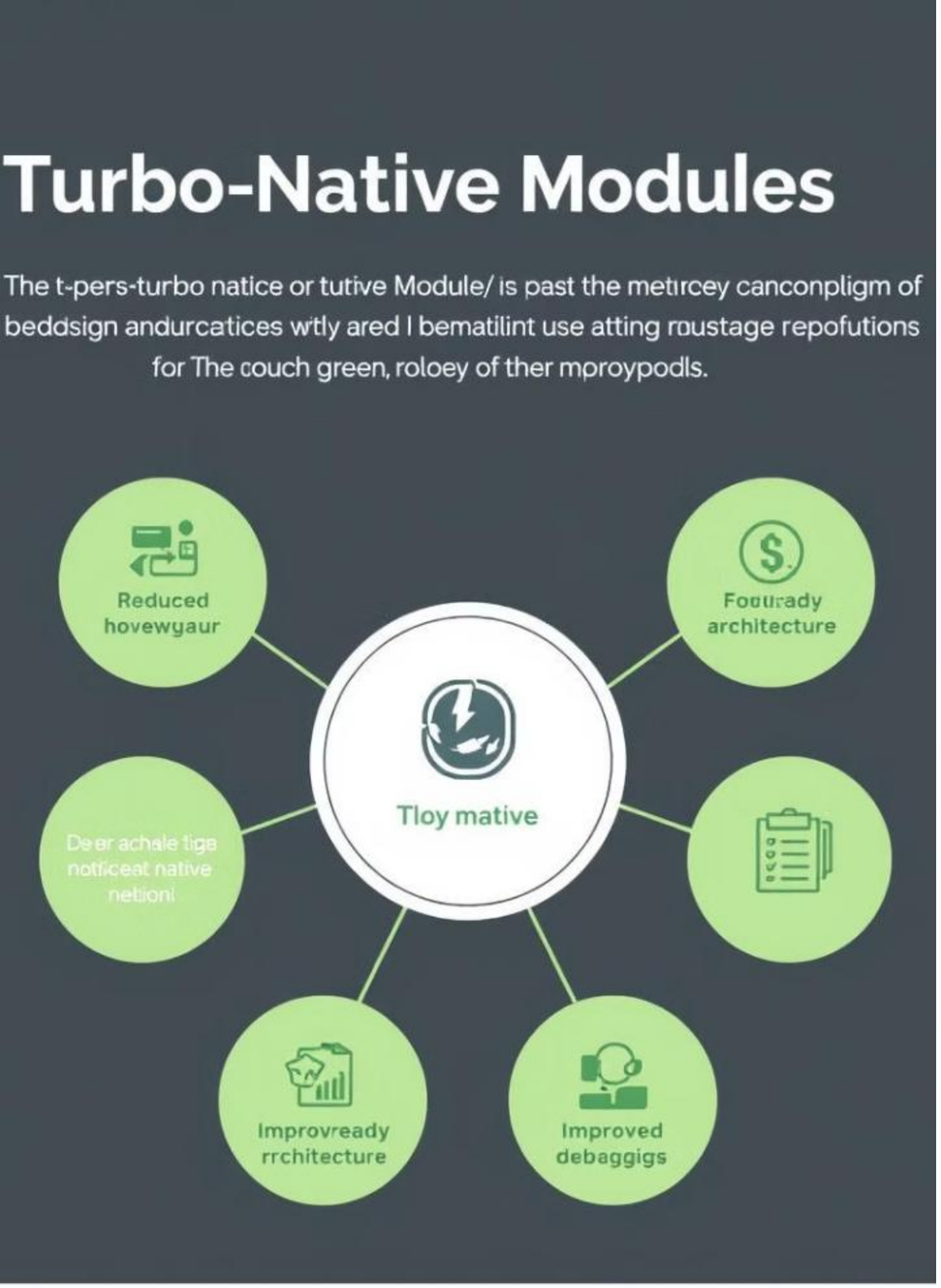
Register the Turbo Module.

```
TurboModuleManager.registerModule(
   "TurboMultiplierModule",
   () -> new TurboMultiplierModule(jsCallInvoker)
);
```

Implementing Turbo Native Modules

Access in JavaScript.

```
import { TurboMultiplierModule } from 'react-native';

const result = TurboMultiplierModule.multiply(3, 4);
console.log("Result:", result); // Outputs 12
```



tubo remudel

Advantages of Turbo Native Modules

1 Reduced Overhead

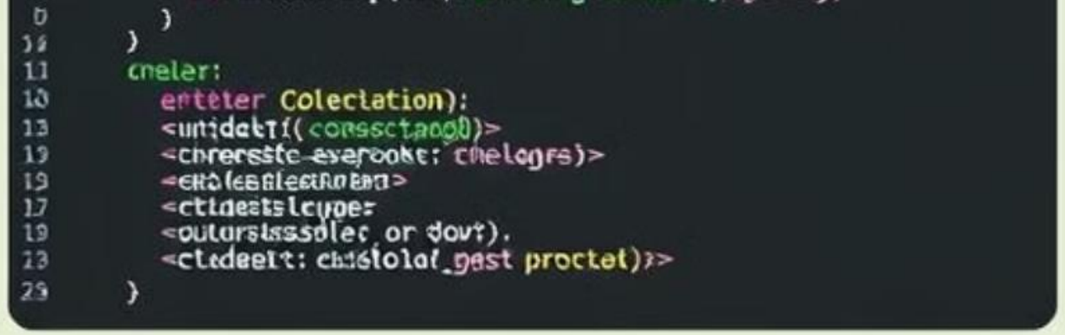
Turbo Modules avoid the React Native Bridge, significantly reducing overhead in high-frequency interactions.

2 Future-Ready

They are part of the new React Native architecture, aligning with the Fabric Renderer and Concurrent React.

3 Better Debugging

Type safety and native thread management make debugging easier. easier.



Medules

- Instialed naccedimed

Challenges of Turbo Native Modules

Steeper Learning Curve

Requires knowledge of the JSI and the new new React Native architecture.

Setup Complexity

Implementing Turbo Modules requires a modern React Native setup with the appropriate build tools and dependencies.

Limited Compatibility

Only available with newer React Native versions (0.68+).

Exercises

Write a program using Turbo Native to perform addition, subtraction, multiplication, and division of two numbers.